

PRACTICAL 1

Deploy Nginx using Docker and Kubernetes

A hands-on lab for learning Docker containers and Kubernetes Deployments, Pods, and Services.

Student details

Field	Details
Name	Lokesh Bhandari
Roll No. / ID	MCA2504151
Batch / Section	SYMCA –B (2025-2027)
Date performed	24 Sep 2026

Learning objectives

By the end of this practical, you should be able to explain and demonstrate:

- What a Kubernetes cluster is
- What a Node is
- What a Pod is
- What a Deployment is
- What a Service is
- How Kubernetes uses Docker / container images
- How to deploy an application on Kubernetes
- How to scale an application
- How to check application logs
- How to delete Kubernetes resources

Before you start

Don't start with Kubernetes YAML, AKS, Helm, Ingress, Terraform, or Jenkins yet. This first practical is done entirely with docker and kubectl commands. Practical 2 will build your own image, push it to Docker Hub, and deploy it with a YAML Deployment and Service — that's where Docker and Kubernetes really connect.

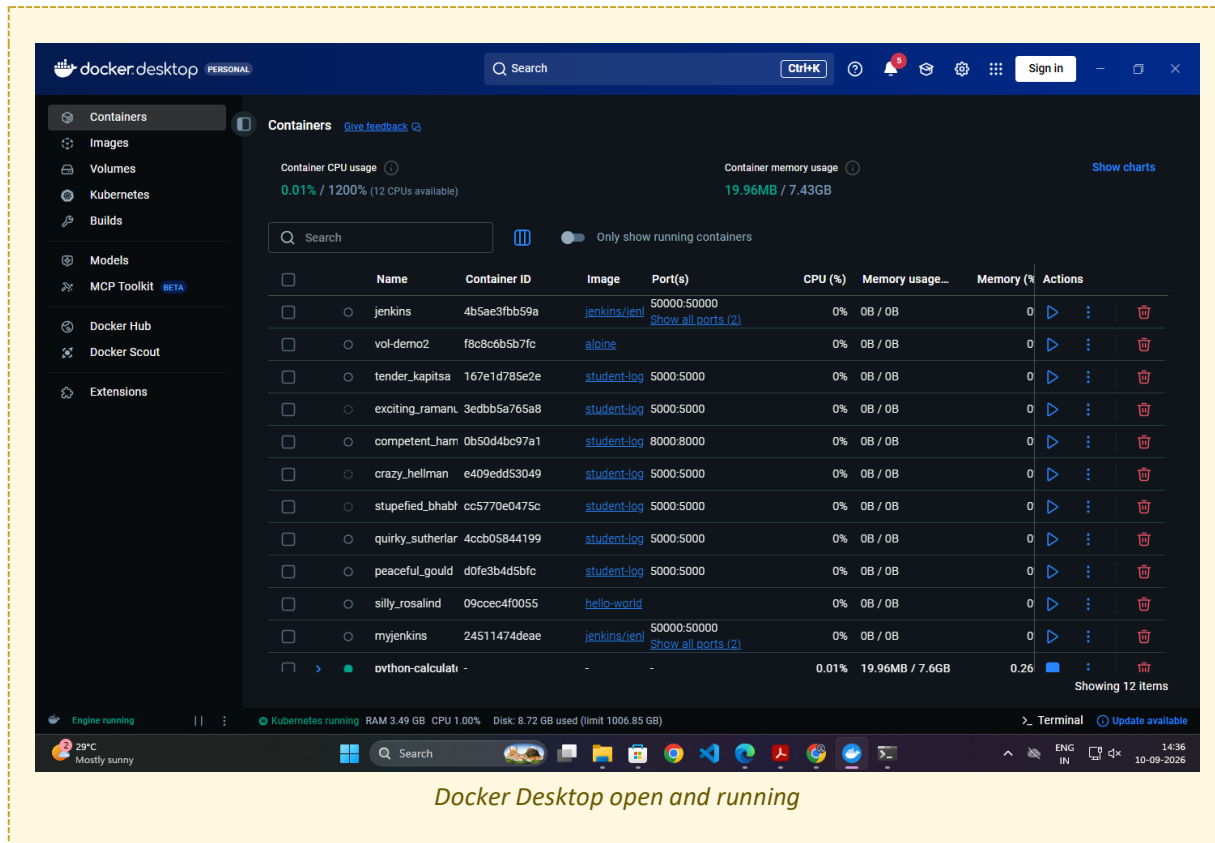
Contents

(Right-click below and choose "Update Field" in Word to generate page numbers.)

Part A — Check Docker

Step A1: Start Docker Desktop

Open Docker Desktop and make sure it is running (the whale icon should be steady, not animating).



Step A2: Check the Docker version

Open Command Prompt or PowerShell and check that Docker is installed correctly.

Run:

```
docker --version
```

Expected output:

```
Docker version 28.x.x
```

Step A3: Check running containers

It is okay if the list is empty — you just want to confirm the command works.

Run:

```
docker ps
```

Expected output:

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
NAMES					

```

C:\Users\IBLAB3>docker --version
Docker version 28.5.1, build e180ab8

C:\Users\IBLAB3>docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
d125215f5bf5   python-calculator-calculator        "python app.py"         9 months ago  Up 10 minutes  0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp  python-calculator

C:\Users\IBLAB3>kubectl version
Client Version: v1.34.1
Kustomize Version: v5.7.1
Unable to connect to the server: net/http: request canceled (Client.Timeout exceeded while awaiting headers)

C:\Users\IBLAB3>kubectl get nodes
E0910 14:26:35.819797    13308 memcache.go:265] "Unhandled Error" err="couldn't get current server API group list: Get \"http://localhost:8080/api?timeout=32s\": net/http: request canceled (Client.Timeout exceeded while awaiting headers)"
E0910 14:27:07.822914    13308 memcache.go:265] "Unhandled Error" err="couldn't get current server API group list: Get \"http://localhost:8080/api?timeout=32s\": context deadline exceeded"
E0910 14:27:39.824364    13308 memcache.go:265] "Unhandled Error" err="couldn't get current server API group list: Get \"http://localhost:8080/api?timeout=32s\": context deadline exceeded (Client.Timeout exceeded while awaiting headers)"
E0910 14:28:11.824557    13308 memcache.go:265] "Unhandled Error" err="couldn't get current server API group list: Get \"http://localhost:8080/api?timeout=32s\": context deadline exceeded"
E0910 14:28:43.825337    13308 memcache.go:265] "Unhandled Error" err="couldn't get current server API group list: Get \"http://localhost:8080/api?timeout=32s\": context deadline exceeded (Client.Timeout exceeded while awaiting headers)"
Unable to connect to the server: context deadline exceeded (Client.Timeout exceeded while awaiting headers)

C:\Users\IBLAB3>kubectl version
Client Version: v1.34.1
Kustomize Version: v5.7.1
Server Version: v1.34.1

C:\Users\IBLAB3>docker --version
Docker version 28.5.1, build e180ab8

C:\Users\IBLAB3>docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
d125215f5bf5   python-calculator-calculator        "python app.py"         9 months ago  Up 22 minutes  0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp  python-calculator

C:\Users\IBLAB3>

```

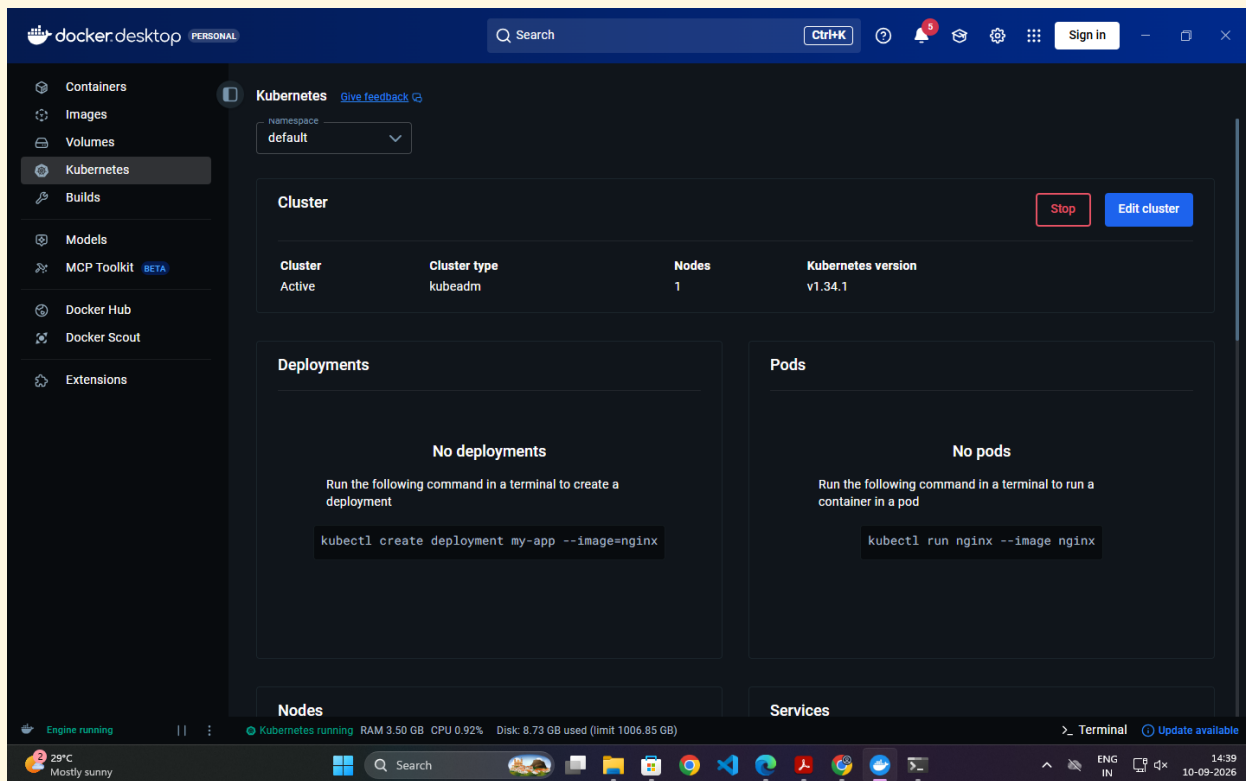
Output of `docker ps` in your terminal

Part B — Enable Kubernetes

In a recent Docker Desktop version: go to Docker Desktop → Kubernetes → Create cluster. Docker Desktop supports local clusters using kubeadm or kind — for this practical, a single-node kubeadm cluster is enough.

In the older interface, instead go to Settings → Kubernetes → Enable Kubernetes, then let Docker Desktop start the cluster.

Wait until Docker Desktop shows the cluster as running before continuing.



Docker Desktop → Kubernetes settings showing the cluster running

Part C — Check Kubernetes

Step C1: Check the kubectl version

Run:

```
kubectl version
```

Step C2: Check cluster nodes

The important word to look for is Ready.

Run:

```
kubectl get nodes
```

Expected output:

NAME	STATUS	ROLES	AGE	VERSION
docker-desktop	Ready	control-plane	...	v1.xx.x

```

C:\Users\IBLAB3>kubectl get nodes
E0910 14:26:35.819797 13308 memcache.go:265] "Unhandled Error" err="couldn't get current server API group list: Get \"http://localhost:8080/api?timeout=32s\": net/http: request canceled (Client.Timeout exceeded while awaiting headers)"
E0910 14:27:07.822914 13308 memcache.go:265] "Unhandled Error" err="couldn't get current server API group list: Get \"http://localhost:8080/api?timeout=32s\": context deadline exceeded"
E0910 14:27:39.824364 13308 memcache.go:265] "Unhandled Error" err="couldn't get current server API group list: Get \"http://localhost:8080/api?timeout=32s\": context deadline exceeded (Client.Timeout exceeded while awaiting headers)"
E0910 14:28:11.824557 13308 memcache.go:265] "Unhandled Error" err="couldn't get current server API group list: Get \"http://localhost:8080/api?timeout=32s\": context deadline exceeded"
E0910 14:28:43.825337 13308 memcache.go:265] "Unhandled Error" err="couldn't get current server API group list: Get \"http://localhost:8080/api?timeout=32s\": context deadline exceeded (Client.Timeout exceeded while awaiting headers)"
Unable to connect to the server: context deadline exceeded (Client.Timeout exceeded while awaiting headers)

C:\Users\IBLAB3>kubectl version
Client Version: v1.34.1
Kustomize Version: v5.7.1
Server Version: v1.34.1

C:\Users\IBLAB3>docker --version
Docker version 28.5.1, build e180ab8

C:\Users\IBLAB3>docker ps
CONTAINER ID   IMAGE                  COMMAND                  CREATED        STATUS        PORTS                               NAMES
d125215f5bf5   python-calculator-calculator   "python app.py"         9 months ago   Up 22 minutes   0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp   python-calculator

C:\Users\IBLAB3>kubectl version
Client Version: v1.34.1
Kustomize Version: v5.7.1
Server Version: v1.34.1

C:\Users\IBLAB3>kubectl get nodes
NAME           STATUS    ROLES    AGE   VERSION
docker-desktop Ready     control-plane   10m   v1.34.1

C:\Users\IBLAB3>

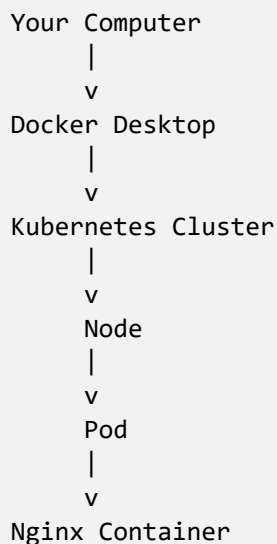
```

Output of `kubectl get nodes` showing STATUS = Ready

If you see Ready, congratulations — you have your first working Kubernetes cluster.

Part D — Understand what you just created

Before doing the practical, understand the layers involved:



Docker vs. Kubernetes

Docker — you directly start a container:

```
docker run nginx
```

Kubernetes — you declare intent, and Kubernetes creates and manages the Pods/containers for you:

```
Deployment
  |
  v
Pod
  |
  v
Nginx Container
```

A Kubernetes Deployment manages Pods and can restart them if necessary; Deployments are also used for scaling applications.

Part E — Download the Nginx Docker image

Step E1: Pull the Nginx image

We'll use Docker directly first, before bringing Kubernetes in.

Run:

```
docker pull nginx
```

Step E2: Confirm the image was downloaded

Run:

```
docker images
```

Expected output:

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
nginx	latest	xxxxxxxxxxxx	...	...

```

3940522783ff86abb131fccff33e48f9c25575ae9785f81dcf055e40bcb68c96
docker: Error response from daemon: ports are not available: exposing port TCP 0.0.0.0:8080 -> 127.0.0.1:0: listen tcp 0.0.0.0:8080: bind: Only one usage of
each socket address (protocol/network address/port) is normally permitted.

Run 'docker run --help' for more information

C:\Users\IBLAB3>docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
Digest: sha256:05b8cb60c354a44ab824ea6e7dc69b46d50762cdbe728a347a5b656e6fb3d7c4
Status: Image is up to date for nginx:latest
docker.io/library/nginx:latest

C:\Users\IBLAB3>docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
nginx               latest             05b8cb60c354       7 days ago         250MB
jenkins/jenkins     lts                f4f65e6cd140       2 months ago       801MB
python-calculator-calculator   latest             7f1f93179eb5       9 months ago       200MB
student-login-app   <none>             96adf0ec0efb       10 months ago      1.6GB
alpine              latest             7b1c37827827       10 months ago      809MB
B                  4b7ce07002c6       11 months ago      12.8M
B
docker/desktop-kubernetes      kubernetes-v1.34.1-cni-v1.7.1-critools-v1.33.0-cri-dockerd-v0.3.20-1-debian  12d6673564e0     11 months ago     591MB
registry.k8s.io/kube-apiserver v1.34.1            b9d7c117f8ac      12 months ago     118MB
registry.k8s.io/kube-controller-manager v1.34.1            2bf47c1b01f5      12 months ago     101MB
registry.k8s.io/kube-scheduler v1.34.1            6e9fbc4e25a5      12 months ago     73.5M
B
registry.k8s.io/kube-proxy     v1.34.1            913cc83ca0b5      12 months ago     102MB
hello-world                   latest             56433a6be3fd      13 months ago     20.3k
B
registry.k8s.io/etcd           3.6.4-0            e36c08168342      13 months ago     273MB
jenkins/jenkins               jdk21              69da3bdc6927      13 months ago     798MB
registry.k8s.io/pause         3.10.1             278fb9dbcca9      14 months ago     1.06M
B
registry.k8s.io/coredns/coredns v1.12.1            e8c262566636      17 months ago     101MB
registry.k8s.io/pause         3.10                ee6521f290b2       2 years ago       1.06M
B
docker/desktop-vpnkit-controller dc331cb22850be0cdd97c84a9cfecaf44alafb6e 7ecf567ea070     3 years ago       47MB
docker/desktop-storage-provisioner v2.0              115d77efe6e2       5 years ago       59.2M
B
C:\Users\IBLAB3>

```

Output of `docker images` showing nginx:latest

Part F — Run Nginx using Docker

Step F1: Run the Nginx container

Understand the command before running it:

docker run — create a container

-d — run in the background

--name my-nginx — name the container my-nginx

-p 8080:80 — map computer port 8080 to container port 80

nginx — the Docker image to use

Run:

```
docker run -d --name my-nginx -p 8080:80 nginx
```

Step F2: Confirm the container is running

Run:

```
docker ps
```

Expected output:

```
... my-nginx ...
```

Step F3: View Nginx in the browser

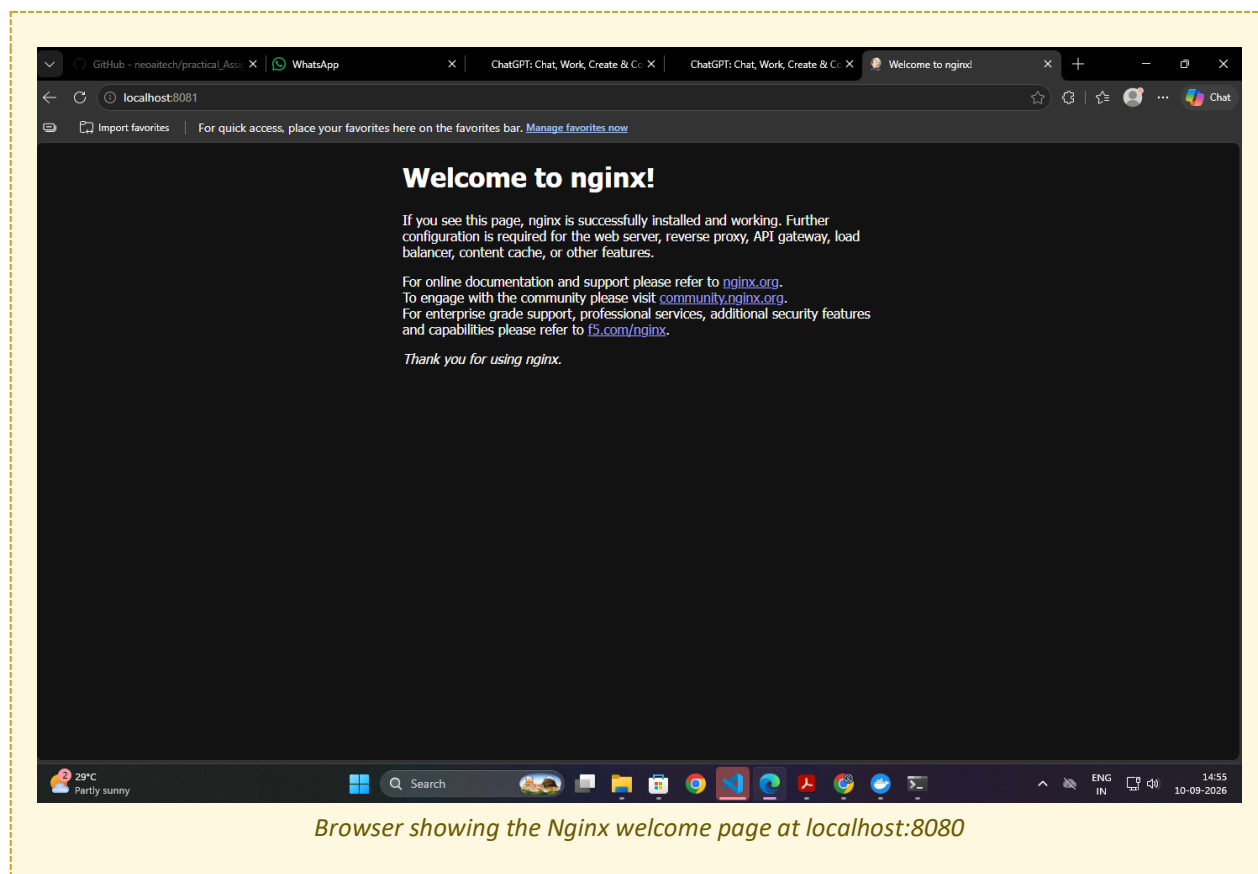
Open your browser and go to the URL below. You should see the default Nginx welcome page.

Run:

```
http://localhost:8080
```

Expected output:

```
Welcome to nginx!
```



You have just deployed Nginx using Docker.

Part G — Stop the Docker container

We don't need this Docker container anymore because Kubernetes will run Nginx from here on.

Step G1: Stop the container

Run:

```
docker stop my-nginx
```

Step G2: Remove the container

Run:

```
docker rm my-nginx
```

Step G3: Confirm it's gone

Run:

```
docker ps
```

Expected output:

```
(my-nginx should no longer appear)
```

Part H — Deploy Nginx using Kubernetes

This is the important part of the practical.

Step H1: Create the Deployment

Run:

```
kubectl create deployment nginx --image=nginx
```

Expected output:

```
deployment.apps/nginx created
```

```

Command Prompt
dcf055e40bcb68c96". You have to remove (or rename) that container to be able to reuse that name.

Run 'docker run --help' for more information

C:\Users\IBLAB3>docker rm -f my-nginx
my-nginx

C:\Users\IBLAB3>docker run -d --name my-nginx -p 8080:80 nginx
b4a58bce84abb3f4d2be35f506b431a888516e6cd0bc6487e06d12821f9015a5
docker: Error response from daemon: ports are not available: exposing port TCP 0.0.0.0:8080 -> 127.0.0.1:0: listen tcp 0.0.0.0:8080: bind: Only one usage of
each socket address (protocol/network address/port) is normally permitted.

Run 'docker run --help' for more information

C:\Users\IBLAB3>docker rm -f my-nginx
my-nginx

C:\Users\IBLAB3>docker run -d --name my-nginx -p 8081:80 nginx
d3205d44beb841835ba0589c8d3d3680b664b0a268ef9768ef911b91fdc055ef5

C:\Users\IBLAB3>docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
d3205d44beb84  nginx         "/docker-entrypoint..." 8 seconds ago  Up 8 seconds  0.0.0.0:8081->80/tcp, [::]:8081->80/tcp  my-nginx
d125215f5bf5   python-calculator-calculator  "python app.py"        9 months ago  Up 39 minutes  0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp  python-calculator

C:\Users\IBLAB3>docker stop my-nginx
my-nginx

C:\Users\IBLAB3>docker rm my-nginx
my-nginx

C:\Users\IBLAB3>docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
d125215f5bf5   python-calculator-calculator  "python app.py"        9 months ago  Up 45 minutes  0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp  python-calculator

C:\Users\IBLAB3>kubectl create deployment nginx --image=nginx
deployment.apps/nginx created

C:\Users\IBLAB3>

```

Terminal output confirming the Deployment was created

You have now created your first Kubernetes Deployment.

Part I — Check the Deployment

Step I1: List Deployments

This means Kubernetes wants 1 Pod, and currently has 1 Pod running.

Run:

```
kubectl get deployments
```

Expected output:

NAME	READY	UP-TO-DATE	AVAILABLE
nginx	1/1	1	1

Part J — Check the Pod

Step J1: List Pods

The random-looking name (e.g. nginx-7c79c4bf97-abcde) is normal — Kubernetes generated it. Don't worry about memorizing it.

Run:

```
kubectl get pods
```

Expected output:

NAME	READY	STATUS	RESTARTS	AGE
nginx-xxxxxxxxx-xxxxx	1/1	Running	0	30s

```

C:\Users\IBLAB3>docker rm -f my-nginx
my-nginx

C:\Users\IBLAB3>docker run -d --name my-nginx -p 8081:80 nginx
d3205d44beb841835ba0589c8d3d3680b664b0a268ef0768ef911b91fdc055ef5

C:\Users\IBLAB3>docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
d3205d44beb8   nginx         "/docker-entrypoint..." 8 seconds ago  Up 8 seconds  0.0.0.0:8081->80/tcp, [::]:8081->80/tcp  my-nginx
d125215f5bf5   python-calculator-calculator  "python app.py"        9 months ago  Up 39 minutes  0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp  python-calculator

C:\Users\IBLAB3>docker stop my-nginx
my-nginx

C:\Users\IBLAB3>docker rm my-nginx
my-nginx

C:\Users\IBLAB3>docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
d125215f5bf5   python-calculator-calculator  "python app.py"        9 months ago  Up 45 minutes  0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp  python-calculator

C:\Users\IBLAB3>kubectl create deployment nginx --image=nginx
deployment.apps/nginx created

C:\Users\IBLAB3>kubectl get deployments
NAME    READY   UP-TO-DATE   AVAILABLE   AGE
nginx   1/1     1            1           61s

C:\Users\IBLAB3>kubectl get pods
NAME                    READY   STATUS    RESTARTS   AGE
nginx-66686b6766-cnwnj  1/1     Running   0           84s

```

Output of `kubectl get pods` showing STATUS = Running

Part K — Understand Deployment and Pod

You now have the following chain of objects:

```

Kubernetes
|
v
Deployment: nginx
|
v

```

```
Pod
 |
 v
Nginx Container
 |
 v
Port 80
```

Object	Meaning
Deployment	Controls the desired number of Pods.
Pod	The basic execution unit in Kubernetes.
Container	Your actual application runs inside the container.

Part L — See the Pod details

Step L1: Describe the Pod

Replace <POD_NAME> with the name you saw in Part J, e.g. nginx-7c79c4bf97-abcde. Look for the Containers: section in the output.

Run:

```
kubectl describe pod <POD_NAME>

# example:
kubectl describe pod nginx-7c79c4bf97-abcde
```

```

C:\Users\IBLAB3>kubectl describe pod nginx-66686b6766-cnwnj
Name:          nginx-66686b6766-cnwnj
Namespace:     default
Priority:       0
Service Account: default
Node:          docker-desktop/192.168.65.3
Start Time:    Thu, 10 Sep 2026 15:00:35 +0530
Labels:        app=nginx
               pod-template-hash=66686b6766
Annotations:   <none>
Status:        Running
IP:            10.1.0.6
IPs:           IP: 10.1.0.6
Controlled By: ReplicaSet/nginx-66686b6766
Containers:
  nginx:
    Container ID:  docker://305d72e6c3ae1dacb2e01fbd22e266cd7c20d7a3b7a71906789b6d97c1492227
    Image:         nginx
    Image ID:      docker-pullable://nginx@sha256:05b8cb60c354a44ab824ea6e7dc69b46d50762cdbe728a347a5b656e6fb3d7c4
    Port:          <none>
    Host Port:     <none>
    State:         Running
      Started:     Thu, 10 Sep 2026 15:00:37 +0530
      Ready:       True
      Restart Count: 0
    Environment:   <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-kmqlw (ro)
Conditions:
  Type              Status
  PodReadyToStartContainers  True
  Initialized        True
  Ready              True
  ContainersReady    True
  PodScheduled       True
Volumes:
  kube-api-access-kmqlw:
    Type:          Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds: 3607
    ConfigMapName:  kube-root-ca.crt
    Optional:      false
  
```

```

Containers:
  nginx:
    Container ID:  docker://305d72e6c3ae1dacb2e01fbd22e266cd7c20d7a3b7a71906789b6d97c1492227
    Image:         nginx
    Image ID:      docker-pullable://nginx@sha256:05b8cb60c354a44ab824ea6e7dc69b46d50762cdbe728a347a5b656e6fb3d7c4
    Port:          <none>
    Host Port:     <none>
    State:         Running
      Started:     Thu, 10 Sep 2026 15:00:37 +0530
      Ready:       True
      Restart Count: 0
    Environment:   <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-kmqlw (ro)
Conditions:
  Type              Status
  PodReadyToStartContainers  True
  Initialized        True
  Ready              True
  ContainersReady    True
  PodScheduled       True
Volumes:
  kube-api-access-kmqlw:
    Type:          Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds: 3607
    ConfigMapName:  kube-root-ca.crt
    Optional:      false
    DownwardAPI:    true
QoS Class:         BestEffort
Node-Selectors:    <none>
Tolerations:       node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                   node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type      Reason      Age   From          Message
  ----      -
  Normal    Scheduled   5m50s  default-scheduler  Successfully assigned default/nginx-66686b6766-cnwnj to docker-desktop
  Normal    Pulling    5m50s  kubelet        Pulling image "nginx"
  Normal    Pulled     5m48s  kubelet        Successfully pulled image "nginx" in 1.85s (1.85s including waiting). Image size: 66343926 bytes.
  Normal    Created    5m48s  kubelet        Created container: nginx
  Normal    Started    5m48s  kubelet        Started container nginx
  
```

Relevant section of `kubectl describe pod` output (Containers:)

Step L2: Get extended Pod info

Run:

```
kubectl get pods -o wide
```

Part M — Check Nginx logs

Step M1: View Pod logs

Run:

```
kubectl logs <POD_NAME>

# example:
kubectl logs nginx-7c79c4bf97-abcde
```

This is the Kubernetes equivalent of the Docker command:

```
docker logs my-nginx      (Docker)
kubectl logs <POD_NAME>  (Kubernetes)
```

Part N — Our application is NOT accessible yet

You created Deployment → Pod → Nginx, but if you try `http://localhost` you should not expect your Kubernetes Nginx to automatically appear there.

Why? Because the Pod only has an internal Kubernetes network address. We need a Service to expose it.

Part O — Create a Kubernetes Service

Step O1: Expose the Deployment

Run:

```
kubectl expose deployment nginx --type=NodePort --port=80
```

Expected output:

```
service/nginx exposed
```

Step O2: List Services

The important part is the port mapping, e.g. `80:30080/TCP`. Your exact NodePort will differ.

Run:

```
kubectl get services
```

Expected output:

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
kubernetes	ClusterIP	10.x.x.x	<none>	443/TCP
nginx	NodePort	10.x.x.x	<none>	80:30xxx/TCP

```

C:\Users\IBLAB3>kubectll logs <POD_NAME>
The syntax of the command is incorrect.

C:\Users\IBLAB3>kubectll logs nginx-66686b6766-cnnwj
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/09/10 09:30:37 [notice] 1#1: using the "epoll" event method
2026/09/10 09:30:37 [notice] 1#1: nginx/1.31.5
2026/09/10 09:30:37 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/09/10 09:30:37 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
2026/09/10 09:30:37 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/09/10 09:30:37 [notice] 1#1: start worker processes
2026/09/10 09:30:37 [notice] 1#1: start worker process 29
2026/09/10 09:30:37 [notice] 1#1: start worker process 30
2026/09/10 09:30:37 [notice] 1#1: start worker process 31
2026/09/10 09:30:37 [notice] 1#1: start worker process 32
2026/09/10 09:30:37 [notice] 1#1: start worker process 33
2026/09/10 09:30:37 [notice] 1#1: start worker process 34
2026/09/10 09:30:37 [notice] 1#1: start worker process 35
2026/09/10 09:30:37 [notice] 1#1: start worker process 36
2026/09/10 09:30:37 [notice] 1#1: start worker process 37
2026/09/10 09:30:37 [notice] 1#1: start worker process 38
2026/09/10 09:30:37 [notice] 1#1: start worker process 39
2026/09/10 09:30:37 [notice] 1#1: start worker process 40

C:\Users\IBLAB3>kubectll expose deployment nginx --type=NodePort --port=80
service/nginx exposed

C:\Users\IBLAB3>kubectll get services
NAME      TYPE      CLUSTER-IP    EXTERNAL-IP    PORT(S)      AGE
kubernetes ClusterIP    10.96.0.1      <none>          443/TCP      41m
nginx     NodePort    10.96.44.135   <none>          80:32345/TCP 16s

```

Output of `kubectll get services` showing the nginx NodePort

Part P — Access Nginx

Because you're using Docker Desktop Kubernetes, exactly how the Service is reachable can depend on your cluster setup.

Step P1: Check the Service

Run:

```
kubectll get service nginx
```

Step P2: Port-forward to the Service

This is the most reliable way to reach the Service from Docker Desktop. Keep this terminal running.

Run:

```
kubectll port-forward service/nginx 8080:80
```

Expected output:

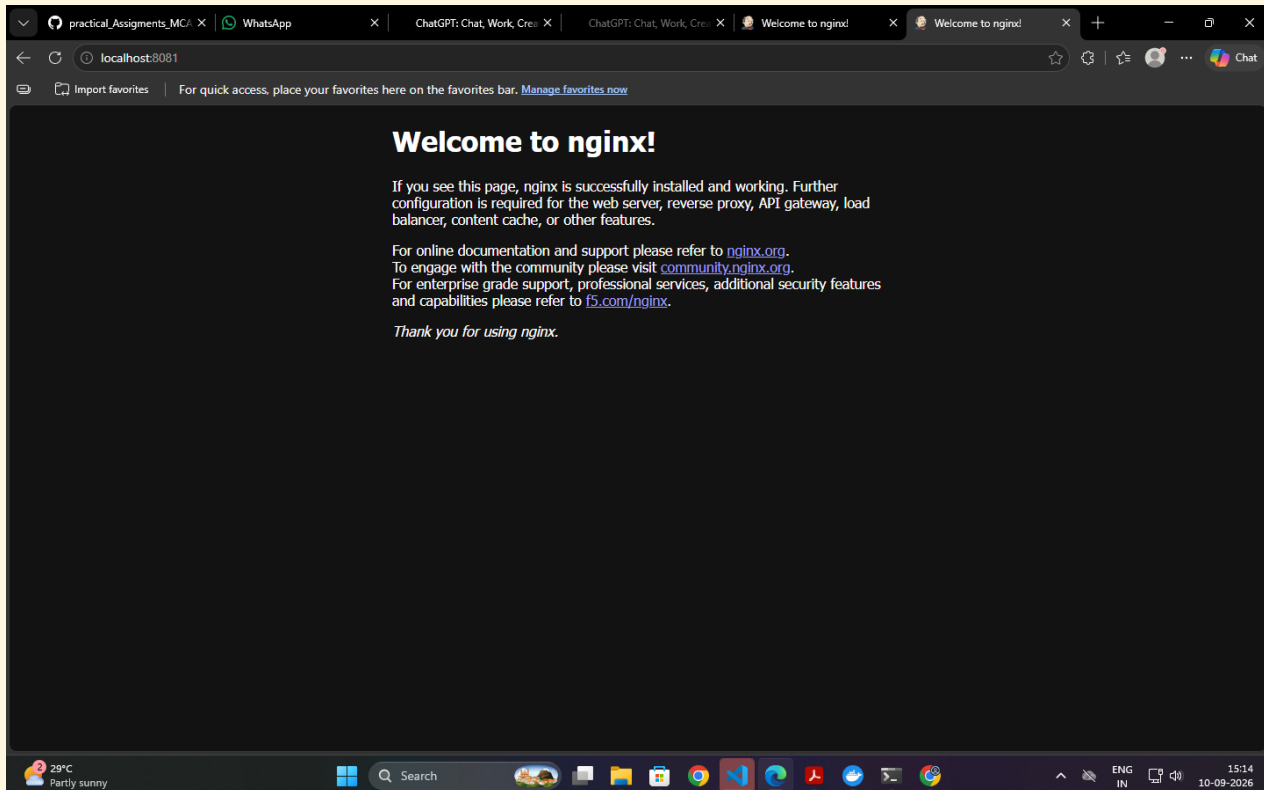
```
Forwarding from 127.0.0.1:8080 -> 80
```

Step P3: Open Nginx in the browser**Run:**

```
http://localhost:8080
```

Expected output:

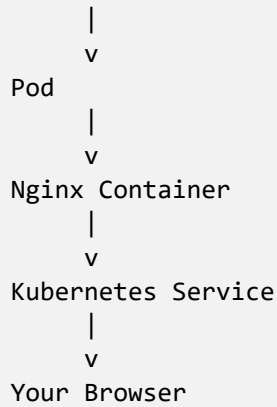
```
Welcome to nginx!
```



Browser showing Nginx served through the Kubernetes Service

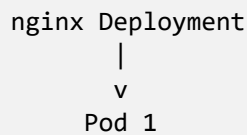
You have now completed the full chain:

```
Docker Image
|
v
Kubernetes Deployment
```



Part Q — The most important experiment: scaling

Currently you have one Pod behind the Deployment:



Step Q1: Scale the Deployment to 3 replicas

Run:

```
kubectl scale deployment nginx --replicas=3
```

Expected output:

```
deployment.apps/nginx scaled
```

Step Q2: Confirm 3 Pods are running

Run:

```
kubectl get pods
```

Expected output:

NAME	READY	STATUS	RESTARTS
nginx-xxxxx-aaaaa	1/1	Running	0
nginx-xxxxx-bbbbb	1/1	Running	0
nginx-xxxxx-ccccc	1/1	Running	0

```

Command Prompt
2026/09/10 09:30:37 [notice] 1#1: start worker process 38
2026/09/10 09:30:37 [notice] 1#1: start worker process 39
2026/09/10 09:30:37 [notice] 1#1: start worker process 40

C:\Users\IBLAB3>kubectl expose deployment nginx --type=NodePort --port=80
service/nginx exposed

C:\Users\IBLAB3>kubectl get services
NAME      TYPE      CLUSTER-IP      EXTERNAL-IP      PORT(S)      AGE
kubernetes ClusterIP  10.96.0.1        <none>           443/TCP       41m
nginx     NodePort   10.96.44.135     <none>           80:32345/TCP  16s

C:\Users\IBLAB3>kubectl get service nginx
NAME      TYPE      CLUSTER-IP      EXTERNAL-IP      PORT(S)      AGE
nginx     NodePort   10.96.44.135     <none>           80:32345/TCP  51s

C:\Users\IBLAB3>kubectl port-forward service/nginx 8080:80
Unable to listen on port 8080: Listeners failed to create with the following errors: [unable to create listener: Error listen tcp4 127.0.0.1:8080: bind: An attempt was made to access a socket in a way forbidden by its access permissions. unable to create listener: Error listen tcp6 [::]:8080: bind: An attempt was made to access a socket in a way forbidden by its access permissions.]
error: unable to listen on any of the requested ports: [{8080 80}]

C:\Users\IBLAB3>kubectl port-forward service/nginx 8081:80
Forwarding from 127.0.0.1:8081 -> 80
Forwarding from [::]:8081 -> 80
Handling connection for 8081
Handling connection for 8081

C:\Users\IBLAB3>

C:\Users\IBLAB3>

C:\Users\IBLAB3>kubectl scale deployment nginx --replicas=3
deployment.apps/nginx scaled

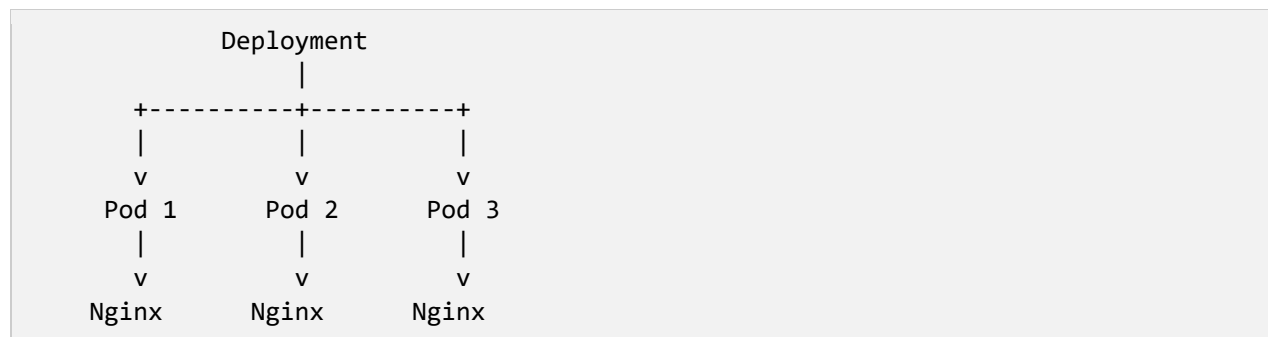
C:\Users\IBLAB3>kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
nginx-66686b6766-4x2qn             1/1     Running   0           18s
nginx-66686b6766-6xdnd             1/1     Running   0           18s
nginx-66686b6766-cnwnj             1/1     Running   0           15m

C:\Users\IBLAB3>

```

Output of `kubectl get pods` showing 3 Running Pods

You now have:



This is one of the fundamental ideas behind Kubernetes.

Part R — Check the Deployment again

Step R1: List the Deployment

3/3 means three Pods are ready out of three desired Pods.

Run:

```
kubectl get deployment
```

Expected output:

NAME	READY	UP-TO-DATE	AVAILABLE
nginx	3/3	3	3

Part 5 — Test Kubernetes self-healing

Step S1: List Pods and copy one Pod name

Run:

```
kubectl get pods
```

Expected output:

(copy one Pod name, e.g. nginx-xxxxx-aaaaa)

Step S2: Delete that Pod

You might think your application is now down.

Run:

```
kubectl delete pod nginx-xxxxx-aaaaa
```

Step S3: Immediately check Pods again

You may see a new Pod already being created.

Run:

```
kubectl get pods
```

Step S4: Check again after a few seconds

You should eventually be back to 3 Pods. Because you told Kubernetes "I want 3 replicas", when it sees Desired = 3 but Actual = 2, it creates another Pod. This is the beginning of self-healing.

Run:

```
kubectl get pods
```

Expected output:

(3 Pods Running again)

```

C:\Users\IBLAB3>kubectl port-forward service/nginx 8081:80
Forwarding from 127.0.0.1:8081 -> 80
Forwarding from [::1]:8081 -> 80
Handling connection for 8081
Handling connection for 8081

C:\Users\IBLAB3>

C:\Users\IBLAB3>kubectl scale deployment nginx --replicas=3
deployment.apps/nginx scaled

C:\Users\IBLAB3>kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
nginx-66686b6766-4x2qn  1/1     Running   0           18s
nginx-66686b6766-6xdnd  1/1     Running   0           18s
nginx-66686b6766-cnwnj  1/1     Running   0           15m

C:\Users\IBLAB3>kubectl get deployment
NAME    READY   UP-TO-DATE   AVAILABLE   AGE
nginx   3/3     3             3           16m

C:\Users\IBLAB3>kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
nginx-66686b6766-4x2qn  1/1     Running   0           108s
nginx-66686b6766-6xdnd  1/1     Running   0           108s
nginx-66686b6766-cnwnj  1/1     Running   0           16m

C:\Users\IBLAB3>kubectl delete pod nginx-xxxxx-aaaaa
Error from server (NotFound): pods "nginx-xxxxx-aaaaa" not found

C:\Users\IBLAB3>kubectl delete pod nginx-66686b6766-4x2qn
pod "nginx-66686b6766-4x2qn" deleted from default namespace

C:\Users\IBLAB3>kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
nginx-66686b6766-6xdnd  1/1     Running   0           4m10s
nginx-66686b6766-7dtbb  1/1     Running   0           62s
nginx-66686b6766-cnwnj  1/1     Running   0           19m

C:\Users\IBLAB3>

```

Before / after: `kubectl get pods` showing the replacement Pod

Part T — View everything together

Step T1: List all resources

This one command is very useful for beginners — it shows Pods, Services, Deployments, and ReplicaSets in one place.

Run:

```
kubectl get all
```

Expected output:

```

pod/...
pod/...
pod/...

service/nginx

deployment.apps/nginx

replicaset.apps/nginx

```

```

Command Prompt
nginx-66686b6766-6xdnd 1/1 Running 0 18s
nginx-66686b6766-cnwnj 1/1 Running 0 15m

C:\Users\IBLAB3>kubectl get deployment
NAME READY UP-TO-DATE AVAILABLE AGE
nginx 3/3 3 3 16m

C:\Users\IBLAB3>kubectl get pods
NAME READY STATUS RESTARTS AGE
nginx-66686b6766-4x2qn 1/1 Running 0 108s
nginx-66686b6766-6xdnd 1/1 Running 0 108s
nginx-66686b6766-cnwnj 1/1 Running 0 16m

C:\Users\IBLAB3>kubectl delete pod nginx-xxxxx-aaaaa
Error from server (NotFound): pods "nginx-xxxxx-aaaaa" not found

C:\Users\IBLAB3>kubectl delete pod nginx-66686b6766-4x2qn
pod "nginx-66686b6766-4x2qn" deleted from default namespace

C:\Users\IBLAB3>kubectl get pods
NAME READY STATUS RESTARTS AGE
nginx-66686b6766-6xdnd 1/1 Running 0 4m10s
nginx-66686b6766-7dtbb 1/1 Running 0 62s
nginx-66686b6766-cnwnj 1/1 Running 0 19m

C:\Users\IBLAB3>kubectl get all
NAME READY STATUS RESTARTS AGE
pod/nginx-66686b6766-6xdnd 1/1 Running 0 4m48s
pod/nginx-66686b6766-7dtbb 1/1 Running 0 100s
pod/nginx-66686b6766-cnwnj 1/1 Running 0 19m

NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
service/kubernetes ClusterIP 10.96.0.1 <none> 443/TCP 49m
service/nginx NodePort 10.96.44.135 <none> 80:32345/TCP 8m41s

NAME READY UP-TO-DATE AVAILABLE AGE
deployment.apps/nginx 3/3 3 3 19m

NAME DESIRED CURRENT READY AGE
replicaset.apps/nginx-66686b6766 3 3 3 19m

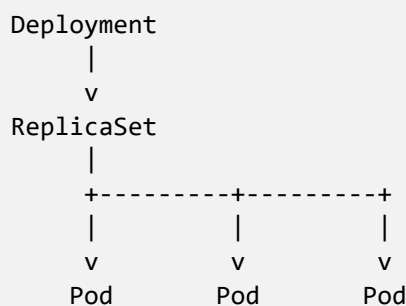
C:\Users\IBLAB3>

```

Full output of `kubectl get all`

Part U — Understand ReplicaSet

You didn't directly create a ReplicaSet — but when you created the Deployment, Kubernetes created one automatically.



Deployment manages the ReplicaSet, and the ReplicaSet manages the desired number of Pods.

Part V — Inspect the Service

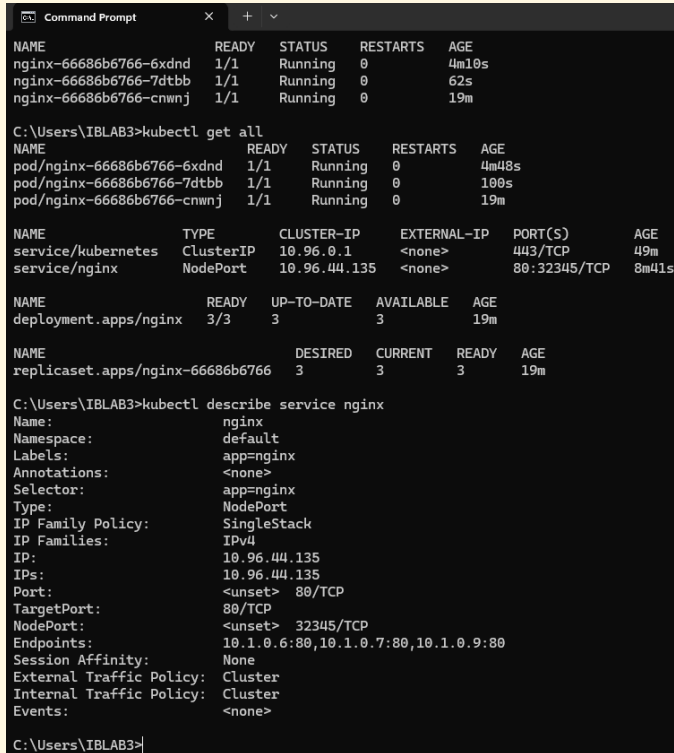
Step V1: Describe the Service

Run:

```
kubectl describe service nginx
```

Expected output:

```
Type: NodePort
Port: 80
TargetPort: 80
Endpoints: ...
```



```
C:\Users\IBLAB3>kubectl get all
NAME                READY   STATUS    RESTARTS   AGE
nginx-66686b6766-6xdnd 1/1     Running   0           4m10s
nginx-66686b6766-7dtbb 1/1     Running   0           62s
nginx-66686b6766-cnwnj 1/1     Running   0           19m

C:\Users\IBLAB3>kubectl get all
NAME                READY   STATUS    RESTARTS   AGE
pod/nginx-66686b6766-6xdnd 1/1     Running   0           4m48s
pod/nginx-66686b6766-7dtbb 1/1     Running   0           100s
pod/nginx-66686b6766-cnwnj 1/1     Running   0           19m

NAME                TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
service/kubernetes  ClusterIP   10.96.0.1    <none>        443/TCP          49m
service/nginx       NodePort    10.96.44.135 <none>        80:32345/TCP     8m41s

NAME                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/nginx 3/3        3            3           19m

NAME                DESIRED   CURRENT   READY   AGE
replicaset.apps/nginx-66686b6766 3          3         3       19m

C:\Users\IBLAB3>kubectl describe service nginx
Name:                 nginx
Namespace:            default
Labels:               app=nginx
Annotations:          <none>
Selector:             app=nginx
Type:                 NodePort
IP Family Policy:     SingleStack
IP Families:          IPv4
IP:                   10.96.44.135
IPs:                  10.96.44.135
Port:                 <unset> 80/TCP
TargetPort:           80/TCP
NodePort:             <unset> 32345/TCP
Endpoints:            10.1.0.6:80,10.1.0.7:80,10.1.0.9:80
Session Affinity:     None
External Traffic Policy: Cluster
Internal Traffic Policy: Cluster
Events:              <none>
```

Relevant section of `kubectl describe service nginx` output

Think of the traffic flow as:

```
Service
|
| port 80
v
Pods
|
| port 80
v
Nginx
```

Part W — Clean up

When your practical is complete, remove the resources you created.

Step W1: Delete the Service

Run:

```
kubectl delete service nginx
```

Step W2: Delete the Deployment

Run:

```
kubectl delete deployment nginx
```

Step W3: Confirm everything is gone

Run:

```
kubectl get all
```

Expected output:

```
(your nginx resources should no longer appear)
```

```

Command Prompt

NAME                TYPE                CLUSTER-IP    EXTERNAL-IP    PORT(S)        AGE
service/kubernetes  ClusterIP           10.96.0.1     <none>         443/TCP        49m
service/nginx       NodePort            10.96.44.135  <none>         80:32345/TCP   8m41s

NAME                READY    UP-TO-DATE    AVAILABLE    AGE
deployment.apps/nginx  3/3      3              3            19m

NAME                DESIRED    CURRENT    READY    AGE
replicaset.apps/nginx-66686b6766  3          3          3        19m

C:\Users\IBLAB3>kubectl describe service nginx
Name:                nginx
Namespace:           default
Labels:              app=nginx
Annotations:          <none>
Selector:             app=nginx
Type:                NodePort
IP Family Policy:    SingleStack
IP Families:         IPv4
IP:                  10.96.44.135
IPs:                 10.96.44.135
Port:                <unset> 80/TCP
TargetPort:          80/TCP
NodePort:            <unset> 32345/TCP
Endpoints:           10.1.0.6:80,10.1.0.7:80,10.1.0.9:80
Session Affinity:    None
External Traffic Policy: Cluster
Internal Traffic Policy: Cluster
Events:              <none>

C:\Users\IBLAB3>kubectl delete service nginx
service "nginx" deleted from default namespace

C:\Users\IBLAB3>kubectl delete deployment nginx
deployment.apps "nginx" deleted from default namespace

C:\Users\IBLAB3>kubectl get all
NAME                TYPE                CLUSTER-IP    EXTERNAL-IP    PORT(S)        AGE
service/kubernetes  ClusterIP           10.96.0.1     <none>         443/TCP        51m

C:\Users\IBLAB3>kubectl delete service nginx

```

Final `kubectl get all` showing a clean cluster

What you should understand after Practical 1

Concept	Simple meaning
Docker Image	Blueprint for a container
Container	Running instance of an image
Kubernetes Cluster	Kubernetes environment
Node	Machine running Kubernetes workloads
Pod	Smallest deployable Kubernetes unit
Deployment	Manages application Pods
ReplicaSet	Maintains desired number of Pods
Service	Provides stable network access to Pods
NodePort	Exposes a Service through a node port
Scaling	Increasing/decreasing number of Pods
Self-healing	Kubernetes recreates failed/deleted Pods

Appendix — Full command list (lab notebook copy)

```

docker --version
docker pull nginx
docker run -d --name my-nginx -p 8080:80 nginx
docker ps
docker stop my-nginx
docker rm my-nginx
kubectl get nodes
kubectl create deployment nginx --image=nginx
kubectl get deployments
kubectl get pods
kubectl get pods -o wide
kubectl describe pod <POD_NAME>
kubectl logs <POD_NAME>
kubectl expose deployment nginx --type=NodePort --port=80
kubectl get services
kubectl port-forward service/nginx 8080:80
kubectl scale deployment nginx --replicas=3
kubectl get pods
kubectl get deployment
kubectl delete pod <POD_NAME>
kubectl get pods
kubectl get all
kubectl delete service nginx
kubectl delete deployment nginx

```